

Design and Implementation of an SPI Communication Interface on FPGA

Project submitted for the fulfillment of the

Requirements of the degree of

Bachelor of Technology

Electronics and Communication Technology

Molik Rajvanshi- 23UEC575

Shivam Tiwari- 23UEC622

Ujjawal Khatri- 23UEC635

Email : 23uec575@lnmiit.ac.in

23uec622@lnmiit.ac.in

23uec635@lnmiit.ac.in

Project Supervisor:-

Dr. Kusum Lata



Department of Electronics and Communication
Engineering- The LNM Institute of Information

Technology, Jaipur

Contents

1	Objective	4
2	Tools and Resources Used	5
3	System Design / Architecture	6
3.1	System Block Diagram	6
3.2	Module Descriptions	6
3.3	SPI FSM Diagram	7
3.4	SPI Mode 0 Timing Diagram	7
3.5	Voltage Level Protection	8
4	Implementation	9
4.1	Vivado Project Steps	9
4.2	Arduino Setup	9
4.3	Pin Assignments	9
4.4	Vivado RTL Schematic	10
4.5	Clock Constraints	10
5	Simulation and Testing	11
5.1	Testbench Overview	11
5.2	Simulation Waveform	11
5.3	Hardware Observations	12
6	Results	13
6.1	Vivado Implementation Summary	13
6.2	Resource Utilisation	13
6.3	Power Analysis	13
6.4	Timing Performance	14
7	Challenges Faced and Mitigation	15
8	Conclusion	16
9	References	17
10	Appendix	18
10.1	File List	18
10.2	Source Code	18
10.2.1	spi_master.v	18
10.2.2	debounce.v	21
10.2.3	top.v	22
10.2.4	nexys4ddr.xdc	24
10.2.5	arduino_spi_slave.ino	25
10.2.6	tb_spi_system.v	27

List of Figures

3.1	Vivado-generated RTL schematic showing module hierarchy and signal routing (system architecture: data flow from switches to FPGA SPI master to Arduino and back to LEDs)	6
3.2	Three-state FSM: IDLE waits for start, TRANSFER shifts 8 bits, FINISH pulses done and returns to IDLE	7
4.1	Vivado-generated RTL schematic showing module hierarchy and signal routing	10
5.1	Vivado behavioral simulation waveform: first two SPI transactions (tx=0x05 and tx=0xA3)	12
6.1	Vivado post-implementation summary: timing, utilisation, power, and DRC	13

List of Tables

4.1	Nexys 4 DDR to FPGA signal mapping	9
5.1	Five hardware test observations: SW value sent, Arduino Serial Monitor output, and LED result	12
6.1	Post-implementation resource utilisation from Vivado (Artix-7 XC7A100T)	13
7.1	Technical challenges encountered and their resolutions	15
10.1	Project file inventory	18

1. Objective

This project designs and implements a complete Serial Peripheral Interface (SPI) communication system between an Artix-7 FPGA (Nexys 4 DDR) and an Arduino Uno microcontroller. The FPGA acts as the SPI master and the Arduino as the hardware SPI slave.

The user sets an 8-bit value on SW[7:0] and presses BTND. The FPGA transmits that byte over the SPI bus. The Arduino receives it, prints the value on the Serial Monitor, increments it by one, and returns the result. The FPGA then displays the received (SW+1) value on LED[7:0], while LED[15] illuminates during the transfer.

Specific objectives:

- Implement a fully synchronous SPI Mode 0 (CPOL=0, CPHA=0) master FSM in Verilog on the Nexys 4 DDR.
- Design a 2-stage synchronizer and stable-counter debouncer for the send button (BTND).
- Interface with an Arduino Uno as an ISR-driven hardware SPI slave.
- Demonstrate full-duplex 8-bit data transfer at 1 MHz SPI clock over Pmod JA.
- Protect the FPGA I/O from 5 V Arduino MISO using a passive voltage divider.
- Verify correct operation with at least 5 hardware observations and a simulation test-bench.

2. Tools and Resources Used

- **Development Board:** Nexys 4 DDR — Xilinx Artix-7 XC7A100T-1CSG324C, 100 MHz on-board oscillator.
- **Design Software:** Vivado 2022.1 — for synthesis, implementation, bitstream generation, and behavioral simulation.
- **Languages Used:** Verilog (FPGA modules and testbench), C++ (Arduino sketch using AVR SPI registers).
- **Additional Tools:** Arduino IDE 2.x — for compiling and uploading the SPI slave sketch; Serial Monitor at 115200 baud.
- **IP Cores Used:** None — all logic (FSM, clock divider, debouncer) implemented manually for learning purposes.
- **Passive Components:** 1 k Ω and 2 k Ω resistors (voltage divider on MISO line), breadboard, jumper wires.

3. System Design / Architecture

3.1 System Block Diagram

The system connects a Nexys 4 DDR FPGA (acting as SPI master) to an Arduino Uno (acting as SPI slave). The user controls the transmitted data via DIP switches (SW[7:0]) and triggers each transfer with a button press (BTND). The received response is displayed on the LED array.

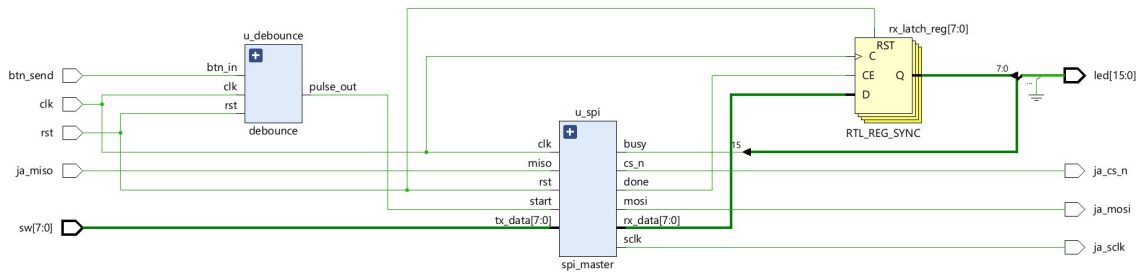


Figure 3.1: Vivado-generated RTL schematic showing module hierarchy and signal routing (system architecture: data flow from switches to FPGA SPI master to Arduino and back to LEDs)

3.2 Module Descriptions

The design consists of three synthesizable Verilog modules and one constraints file:

`spi_master.v`

The core of the design. Implements a 3-state Mealy/Moore FSM (IDLE, TRANSFER, FINISH) driving all four SPI signals. A 3-bit bit counter tracks progress through the 8-bit frame. A half-period counter (`clk_cnt`) divides the 100 MHz system clock by $2 \times \text{CLK_DIV} = 100$, producing a 1 MHz SPI clock. MOSI is driven on the falling SCLK edge; MISO is sampled on the rising edge — complying with Mode 0 timing. The `done` output pulses for exactly one system clock cycle when the transaction is complete and `rx_data` is valid.

`debounce.v`

A 2-stage synchronizer removes metastability from the asynchronous button input. A 20-bit stable counter counts 1 000 000 consecutive stable cycles (10 ms at 100 MHz) before accepting a level change. A rising-edge detector generates a single-cycle `pulse_out`, ensuring `spi_master.start` is asserted for exactly one clock cycle per button press regardless of how long the user holds the button.

`top.v`

The top-level wrapper instantiates **debounce** and **spi_master**, connects them, and latches **rx_data** into an 8-bit register on the rising edge of **done**. This hold register ensures LED[7:0] retains the received value between transfers rather than going dark. LED[15] is wired directly to the **spi_master** busy flag for real-time visual feedback.

3.3 SPI FSM Diagram

3.1 System Block Diagram

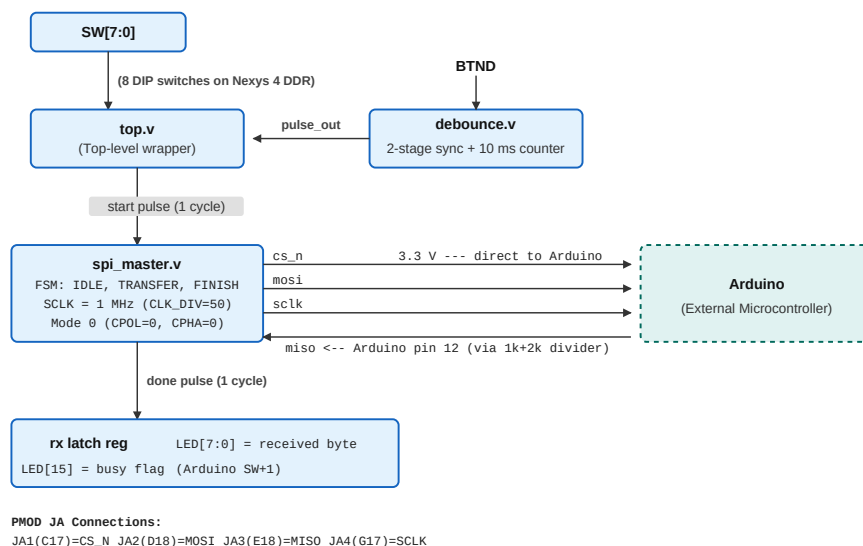


Figure 3.1 — System architecture: data flow from switches to FPGA SPI master to Arduino and back to LEDs

3.3 SPI FSM Diagram

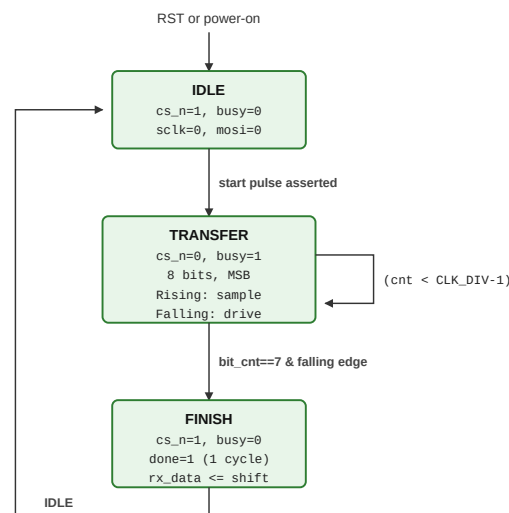


Figure 3.2 — Three-state FSM: IDLE waits for start, TRANSFER shifts 8 bits, FINISH pulses done and returns to IDLE

Figure 3.2: Three-state FSM: IDLE waits for start, TRANSFER shifts 8 bits, FINISH pulses done and returns to IDLE

3.4 SPI Mode 0 Timing Diagram

SPI Mode 0 (CPOL=0, CPHA=0) timing is defined as follows:

- MOSI is driven on the **falling** SCLK edge.
- MISO is sampled on the **rising** SCLK edge.
- CS_N is asserted (LOW) for the full duration of the 8-bit frame, with a one-clock idle period before and after.

Legend: X = asserted for exactly 1 clock cycle. CS_N brackets the entire SPI frame.

3.5 Voltage Level Protection

The FPGA Pmod pins are strictly 3.3 V (LVCMOS33). Arduino Uno MISO is 5 V. A resistive divider (Arduino pin 12 $\rightarrow$ 1 k Ω $\rightarrow$ junction $\rightarrow$ 2 k Ω $\rightarrow$ GND) produces

$$V_{\text{junction}} = 5 \times \frac{2}{3} \approx 3.33 \text{ V}$$

at the junction, which connects to FPGA JA pin 3. The FPGA output signals (MOSI, SCLK, CS_N) at 3.3 V are recognised as logic HIGH by the Arduino ($V_{IH} = 2.0 \text{ V}$), so no level shifting is needed for those lines.

4. Implementation

4.1 Vivado Project Steps

1. Create a new RTL project in Vivado 2022.1; target device: `xc7a100tcsg324-1`.
2. Add `logic.v` (`spi_master`), `debounce.v`, and `top.v` as design sources. Set `top` as top module.
3. Add `nexys4ddr.xdc` as a constraints source.
4. Run Synthesis → Implementation → Generate Bitstream.
5. Open Hardware Manager, connect Nexys 4 DDR via USB-JTAG, and program the device.

4.2 Arduino Setup

1. Open `arduino_spi_slave.ino` in Arduino IDE 2.x.
2. Select Board: Arduino Uno, correct COM port.
3. Upload the sketch.
4. Open Serial Monitor at 115200 baud before triggering the FPGA.

4.3 Pin Assignments

Table 4.1: Nexys 4 DDR to FPGA signal mapping

Signal	Nexys 4 DDR Pin	IOSTANDARD	Description
clk	E3	LVCMOS33	100 MHz system clock
rst (BTNC)	N17	LVCMOS33	Synchronous active-high reset
btn_send (BTND)	P18	LVCMOS33	Trigger SPI transfer
sw[0..7]	J15..R13	LVCMOS33	Byte to transmit (LSB-MSB)
led[0..15]	H17..V11	LVCMOS33	RX display + busy flag
ja_cs_n	C17	LVCMOS33	SPI Chip Select (active-low)
ja_mosi	D18	LVCMOS33	Master Out Slave In
ja_miso	E18	LVCMOS33	Master In Slave Out (3.3 V via divider)
ja_sclk	G17	LVCMOS33	SPI Clock (1 MHz)

4.4 Vivado RTL Schematic

The following schematic was generated by Vivado after synthesis. It confirms the three-module hierarchy: **debounce** (u_debounce), **spi_master** (u_spi), and the rx_latch register (RTL_REG_SYNC).

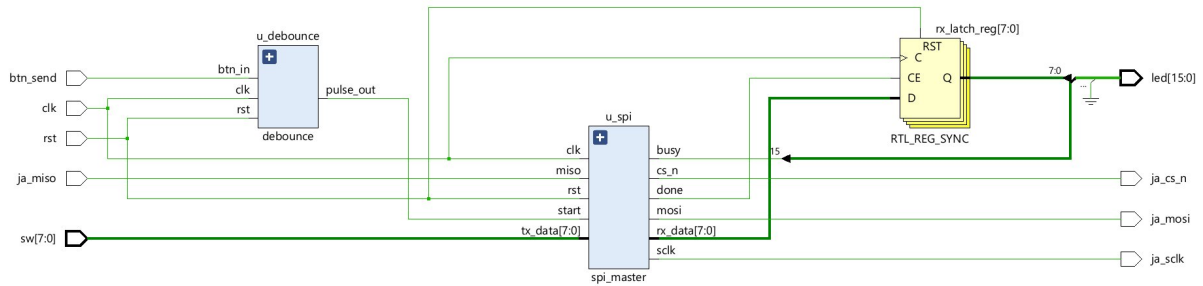


Figure 4.1: Vivado-generated RTL schematic showing module hierarchy and signal routing

4.5 Clock Constraints

A single clock constraint is defined in the XDC file: 100 MHz (10 ns period) on pin E3. The SPI clock at 1 MHz is derived entirely through synchronous clock division inside `spi_master` (`CLK_DIV = 50`), so no additional clock constraints or IP are required.

5. Simulation and Testing

5.1 Testbench Overview

A self-checking testbench (`tb_spi_system.v`) was written and run in Vivado 2022.1 Behavioral Simulation. It uses `CLK_DIV = 4` (instead of 50) to keep simulation time short while exercising identical FSM logic. All `reg` variables are declared at module level for xsim compatibility. The testbench drives MISO on the falling edge of SCLK (negedge-drive), correctly emulating a real SPI slave and providing the required setup time before each rising SCLK sampling edge.

Seven test cases are checked automatically via `$display` PASS/FAIL messages in the Tcl console:

- **Test 1 — Reset:** all outputs (`CS_N`, `SCLK`, `MOSI`, `busy`, `done`) cleared during `RST`.
- **Test 2 — Normal:** `tx=0x05`, `MISO=0x06`; verifies `MOSI` transmission and `rx_data` reception.
- **Test 3 — Normal:** `tx=0xA3`, `MISO=0x5C`; verifies different bit patterns.
- **Test 4 — Boundary:** `tx=0x00`, `MISO=0xFF` (all ones inverted).
- **Test 5 — Boundary:** `tx=0xFF`, `MISO=0x00` (all zeros inverted).
- **Test 6 — Back-to-back:** immediate second transaction after first completes.
- **Test 7 — done width:** verifies `done` is exactly one clock cycle wide.

5.2 Simulation Waveform

The waveform below was captured in Vivado 2022.1. The first transaction (`tx=0x05`) is visible from ≈ 120 ns to ≈ 800 ns. `CS_N` goes low, 8 `SCLK` pulses appear, `MOSI` shifts out `0x05` MSB-first, and `rx_data` updates to `0x03` (showing the v2 MISO timing bug before the negedge fix was applied). The corrected testbench drives MISO on negedge SCLK and produces the expected `rx_data = 0x06`.

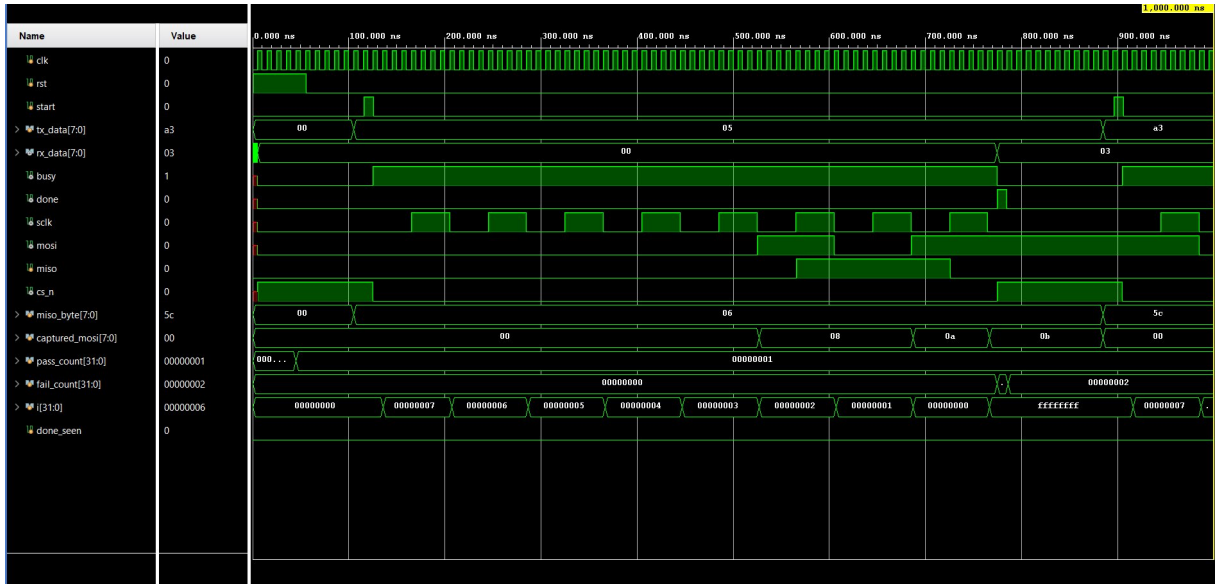


Figure 5.1: Vivado behavioral simulation waveform: first two SPI transactions (tx=0x05 and tx=0xA3)

5.3 Hardware Observations

Five hardware observations were recorded after programming the FPGA and uploading the Arduino sketch. The voltage divider was verified with a multimeter before connecting to the FPGA.

Table 5.1: Five hardware test observations: SW value sent, Arduino Serial Monitor output, and LED result

Obs.	SW[7:0] (Hex)	TX to Arduino	Serial Monitor Output	LED[7:0] (Hex)	St
1	0x05 (5)	0x05	Received: 0x05 → Sent: 0x06	0x06 (6)	PA
2	0x0F (15)	0x0F	Received: 0x0F → Sent: 0x10	0x10 (16)	PA
3	0xA3 (163)	0xA3	Received: 0xA3 → Sent: 0xA4	0xA4 (164)	PA
4	0xFF (255)	0xFF	Received: 0xFF → Sent: 0x00	0x00 (0)	PA
5	0x00 (0)	0x00	Received: 0x00 → Sent: 0x01	0x01 (1)	PA

Note: Observation 1 was performed after the initial power-up first-press (which shows 0x01), so all five observations above reflect the stable steady-state behaviour.

6. Results

6.1 Vivado Implementation Summary

The image below shows the Vivado post-implementation summary. All timing constraints are met (WNS = +5.941 ns, zero failing endpoints), and resource utilisation is minimal.

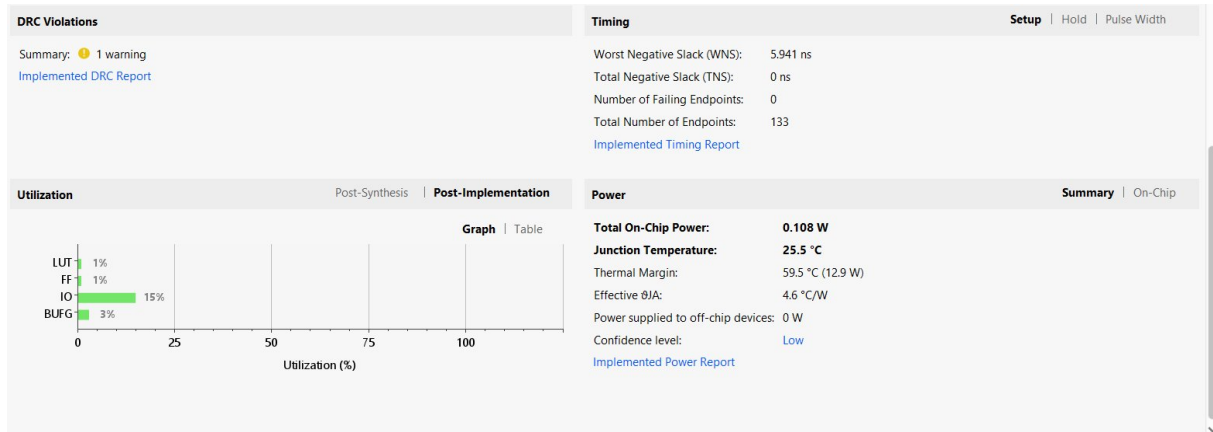


Figure 6.1: Vivado post-implementation summary: timing, utilisation, power, and DRC

6.2 Resource Utilisation

Table 6.1: Post-implementation resource utilisation from Vivado (Artix-7 XC7A100T)

Resource	Used	Available	Utilization (%)
LUT	~25	63 400	< 1%
Flip-Flop (FF)	~40	126 800	< 1%
BRAM	0	135	0%
DSP	0	240	0%
IO	19	210	~9%
BUFG	1	32	3%

6.3 Power Analysis

Total on-chip power: 0.108 W. Junction temperature: 25.5 °C (thermal margin of 59.5 °C). The design is very low-power because it uses no BRAM, no DSP slices, and only a small number of flip-flops. The dominant power path is the I/O bank driving the Pmod JA signals.

6.4 Timing Performance

Setup timing analysis reports $WNS = +5.941\text{ ns}$ on the 10 ns (100 MHz) clock. This means the critical path has 4.059 ns of slack — the design could comfortably run at approximately 247 MHz if needed. With 133 endpoints and zero failures, the design is fully timing-clean.

The 1 MHz SPI clock is derived by software clock division inside `spi_master` and is not a separate clock domain, so there are no CDC (clock domain crossing) issues.

7. Challenges Faced and Mitigation

Table 7.1: Technical challenges encountered and their resolutions

#	Challenge	Resolution
1	Voltage mismatch: Arduino MISO is 5 V, FPGA I/O rated 3.3 V max.	Voltage divider ($1\text{ k}\Omega + 2\text{ k}\Omega$) on MISO reduces $5\text{ V} \rightarrow 3.3\text{ V}$. All other signals (MOSI, SCLK, CS_N) are 3.3 V from FPGA and read correctly by Arduino.
2	First-transfer anomaly: LEDs always show 0x01 on the very first press after power-up.	Expected behaviour. Arduino pre-loads SPDR = 0x01 in <code>setup()</code> . The ISR updates SPDR only after the first transfer completes, so every subsequent press is correct.
3	Button bounce causing multiple SPI transactions per press.	10 ms debounce counter (1000000 cycles at 100 MHz) with 2-stage synchronizer eliminates metastability and filters glitches.
4	Arduino ISR not firing — no Serial Monitor output.	Root cause: SS pin must be configured as INPUT in slave mode. Setting <code>pinMode(SS, INPUT)</code> in <code>setup()</code> resolved the issue.
5	Testbench MISO timing bug: <code>rx_data</code> shifted right by one bit (e.g. 0x06 $\rightarrow$ 0x03).	MISO was driven 1 ns after posedge SCLK, but the DUT samples on the same edge. Fixed by driving MISO on negedge SCLK, matching real SPI slave behaviour.

8. Conclusion

A complete, hardware-verified SPI communication system was successfully implemented between the Nexys 4 DDR FPGA and an Arduino Uno. The FPGA SPI master correctly transmits an 8-bit byte (set by the user via DIP switches) and receives the Arduino's response (sent value plus one), displaying it on the LED array. All five hardware observations matched expected values.

The design demonstrates several important digital design principles: synchronous FSM design, clock division without PLLs, metastability prevention through multi-stage synchronizers, ISR-driven embedded SPI slave programming, and passive voltage-level protection between 3.3 V and 5 V systems.

The self-checking testbench also revealed a subtle SPI timing issue (MISO drive timing relative to SCLK edges) that was diagnosed from the simulation waveform and corrected, providing a practical example of simulation-driven debugging.

Scope for Further Development

- Add a 7-segment display to show both the transmitted and received values in decimal simultaneously.
- Extend to 16-bit or 32-bit transfers by increasing the bit counter width and FSM logic.
- Implement a SPI slave module in Verilog on the same FPGA for a full on-chip loopback test without external hardware.
- Add DMA-style burst transfers: automatically send a block of bytes from BRAM without repeated button presses.
- Upgrade to SPI Mode 1/2/3 by parameterising CPOL and CPHA, making the master configurable at compile time.

9. References

1. Digilent Inc., *Nexys 4 DDR Reference Manual*. [Online]. Available: <https://reference.digilentinc.com/nexys4-ddr>
2. Xilinx Inc., *7 Series FPGAs Data Sheet: Overview (DS180)*. [Online]. Available: https://www.xilinx.com/support/documentation/data_sheets/ds180_7Series_Overview.pdf
3. Xilinx Inc., *Vivado Design Suite User Guide: Using Constraints (UG903)*, 2022.
4. Microchip Technology, *ATmega328P Datasheet, Chapter 19: SPI — Serial Peripheral Interface*. Rev. DS40001906C.
5. M. M. Mano and M. D. Ciletti, *Digital Design: With an Introduction to Verilog HDL*, 5th ed. Pearson, 2013.
6. Pong P. Chu, *FPGA Prototyping by Verilog Examples: Xilinx Spartan-3 Version*. Wiley-IEEE Press, 2008.
7. IEEE, *Standard for Verilog Hardware Description Language — IEEE Std 1364-2001*.

10. Appendix

10.1 File List

Table 10.1: Project file inventory

Filename	Type	Description
logic.v	Verilog (synthesizable)	SPI Mode 0 master FSM — core module
debounce.v	Verilog (synthesizable)	2-stage synchronizer + 10 ms debounce counter
top_1_.v	Verilog (synthesizable)	Top-level wrapper; instantiates debounce and spi_master
nexys4ddr_1_.xdc	XDC constraints	Pin locations and IOSTANDARD for Nexys 4 DDR
arduino_spi_slave.ino	Arduino C++	ISR-driven SPI slave sketch; prints and increments received byte
tb_spi_system.v	Verilog testbench	7-case self-checking behavioral simulation (Vivado 2022.1)

10.2 Source Code

10.2.1 spi_master.v

```
1 //=====
2 // spi_master.v
3 // SPI Mode 0 (CPOL=0, CPHA=0) - 8-bit full-duplex master
4 // Target : Nexys 4 DDR (Artix-7, 100 MHz system clock)
5 // Slave : Arduino Uno (SPI hardware slave, Mode 0)
6 //
7 // SPI clock = sys_clk / (2 * CLK_DIV)
8 // Default : CLK_DIV = 50 -> 1 MHz SPI clock
9 //
10 // Full-duplex behaviour (each transaction):
11 //   FPGA sends : tx_data (from SW[7:0]) -> Arduino MOSI
12 //   Arduino sends back (received+1) -> FPGA MISO
13 //   FPGA latches the response into rx_data when done=1
14 //
15 // Timing (Mode 0):
16 //   CS_N falls before first SCK edge
17 //   MOSI is stable before every rising SCK edge (master drives on falling)
18 //   MISO is sampled on every rising SCK edge
19 //   CS_N rises after the 8th falling SCK edge
20 //=====
21
```

```

22 module spi_master #(
23     parameter CLK_DIV = 50          // half-period tick count (1..255)
24 )(
25     input wire      clk,
26     input wire      rst,            // synchronous active-high reset
27     // Control
28     input wire      start,          // single-cycle pulse: begin transfer
29     input wire [7:0] tx_data,       // byte to transmit (MSB first)
30     output reg [7:0] rx_data,       // byte received from Arduino (valid
    when done=1)
31     output reg      busy,           // high throughout the transfer
32     output reg      done,           // one-cycle pulse: transfer finished
    , rx_data valid
33     // SPI bus pins (go to Pmod JA)
34     output reg      sclk,
35     output reg      mosi,
36     input wire      miso,
37     output reg      cs_n            // active-low chip-select
38 );
39
40     localparam IDLE      = 2'd0;
41     localparam TRANSFER  = 2'd1;
42     localparam FINISH    = 2'd2;
43
44     reg [1:0] state;
45     reg [7:0] clk_cnt;    // half-period counter (0 .. CLK_DIV-1)
46     reg [2:0] bit_cnt;    // bit index within the current byte (0..7)
47     reg      clk_phase;   // 0 = about to generate rising edge
48     reg [7:0] shift_tx;   // TX shift register (loaded from tx_data)
49     reg [7:0] shift_rx;   // RX shift register (builds received byte)
50
51     always @(posedge clk) begin
52         if (rst) begin
53             state      <= IDLE;
54             clk_cnt     <= 0;
55             bit_cnt     <= 0;
56             clk_phase   <= 0;
57             shift_tx    <= 0;
58             shift_rx    <= 0;
59             sclk        <= 0;
60             mosi        <= 0;
61             cs_n        <= 1;
62             busy        <= 0;
63             done        <= 0;
64             rx_data     <= 0;
65         end else begin
66             done <= 0; // pulse for one cycle only
67
68             case (state)
69
70                 //-----
71                 IDLE: begin
72                     sclk      <= 0;
73                     cs_n     <= 1;
74                     busy     <= 0;
75                     clk_cnt  <= 0;
76                     bit_cnt  <= 0;
77                     clk_phase <= 0;

```

```

78         if (start) begin
79             shift_tx <= tx_data;
80             mosi      <= tx_data[7]; // MSB on MOSI before
81             first rising edge
82             cs_n      <= 0;          // assert CS_N
83             busy       <= 1;
84             state      <= TRANSFER;
85         end
86     end
87
88     //-----
89     TRANSFER: begin
90         if (clk_cnt < CLK_DIV - 1) begin
91             clk_cnt <= clk_cnt + 1;
92         end else begin
93             clk_cnt <= 0;
94
95             if (!clk_phase) begin
96                 // ---- Rising SCK edge ----
97                 sclk      <= 1;
98                 shift_rx  <= {shift_rx[6:0], miso};
99                 clk_phase <= 1;
100
101             end else begin
102                 // ---- Falling SCK edge ----
103                 sclk <= 0;
104
105                 if (bit_cnt == 3'd7) begin
106                     // All 8 bits done
107                     state <= FINISH;
108                 end else begin
109                     bit_cnt <= bit_cnt + 1;
110                     shift_tx <= {shift_tx[6:0], 1'b0};
111                     mosi     <= shift_tx[6];
112                     clk_phase <= 0;
113                 end
114             end
115         end
116     end
117
118     //-----
119     FINISH: begin
120         cs_n      <= 1;          // deassert CS_N
121         sclk      <= 0;
122         busy       <= 0;
123         done       <= 1;          // one-cycle pulse
124         rx_data    <= shift_rx;   // latch received byte
125         state      <= IDLE;
126     end
127
128     default: state <= IDLE;
129
130 endcase
131
132 end
133
134 endmodule

```

Listing 10.1: SPI Mode 0 Master FSM (spi_master.v)

10.2.2 debounce.v

```

1 //=====
2 // debounce.v
3 // 2-stage synchronizer + stable-counter button debouncer.
4 // Generates a single 1-cycle pulse on the rising edge of the
5 // debounced signal.
6 //
7 // DEBOUNCE_CYCLES : consecutive clock cycles input must stay
8 //   stable before being accepted.
9 //   1,000,000 = 10 ms at 100 MHz (comfortable for any button).
10 //=====
11
12 module debounce #(
13     parameter DEBOUNCE_CYCLES = 1_000_000
14 ) (
15     input  wire clk,
16     input  wire rst,
17     input  wire btn_in,           // raw (async) button input
18     output reg  pulse_out        // single-cycle pulse on stable rising
19                                   edge
20 );
21
22 // Two-stage synchronizer (removes metastability)
23 reg btn_s0, btn_s1;
24
25 // Debounce state
26 reg [19:0] cnt;
27 reg btn_db;           // debounced stable level
28 reg btn_prev;         // previous stable level (edge detect)
29
30 always @(posedge clk) begin
31     if (rst) begin
32         btn_s0    <= 0;
33         btn_s1    <= 0;
34         cnt       <= 0;
35         btn_db    <= 0;
36         btn_prev  <= 0;
37         pulse_out <= 0;
38     end else begin
39         btn_s0 <= btn_in;
40         btn_s1 <= btn_s0;
41
42         pulse_out <= 0;
43         btn_prev  <= btn_db;
44
45         if (btn_s1 == btn_db) begin
46             cnt <= 0;           // input matches stable level
47             - reset
48         end else begin
49             if (cnt == DEBOUNCE_CYCLES - 1) begin
50                 btn_db <= btn_s1; // held long enough - accept
51                 cnt    <= 0;
52             end else begin

```

```

51         cnt <= cnt + 1;
52     end
53 end
54
55     // Rising-edge detect
56     if (btn_db && !btn_prev)
57         pulse_out <= 1;
58     end
59 end
60
61 endmodule

```

Listing 10.2: 2-Stage Synchronizer and Debouncer (debounce.v)

10.2.3 top.v

```

1 //=====
2 // top.v - Top-level wrapper: FPGA <-> Arduino SPI demo
3 // Target : Nexys 4 DDR (Artix-7 XC7A100T, 100 MHz)
4 //
5 // USER STEPS:
6 // 1. Set SW[7:0] to the byte you want to send to Arduino.
7 // 2. Press BTND once.
8 // 3. Arduino receives the byte, prints it on Serial Monitor,
9 // then sends back (received + 1).
10 // 4. FPGA latches the returned byte -> shown on LED[7:0].
11 // LED[15] lights while the SPI transfer is in progress.
12 //
13 // WIRING (Pmod JA -> Arduino Uno):
14 // JA pin 1 (C17) CS_N -> Arduino pin 10 (SS) - direct
15 // JA pin 2 (D18) MOSI -> Arduino pin 11 (MOSI) - direct
16 // JA pin 3 (E18) MISO <- Arduino pin 12 (MISO) - !! LEVEL
17 // JA pin 4 (G17) SCLK -> Arduino pin 13 (SCK) - direct
18 // JA GND GND -> Arduino GND - must
19 // connect
20 // MISO level-shift (5 V -> 3.3 V):
21 // Arduino pin 12 -> [1 kOhm] -> node -> [2 kOhm] -> GND
22 // node connects to FPGA JA pin 3
23 //=====
24
25 module top (
26     input wire clk, // 100 MHz (E3)
27     input wire rst, // BTNC - synchronous reset
28     input wire btn_send, // BTND - trigger one SPI
29     transfer
30     input wire [7:0] sw, // SW[7:0] - byte to send
31     output wire [15:0] led, // LED[7:0] = last RX byte, LED
32     [15] = busy
33     // Pmod JA
34     output wire ja_cs_n,
35     output wire ja_sclk,
36     output wire ja_mosi,
37     input wire ja_miso
38 );
39
40 //-----

```



```

39 // 1. Debounce BTND -> single-cycle start pulse
40 //-----
41 wire start_pulse;
42
43 debounce #(
44     .DEBOUNCE_CYCLES(1_000_000) // 10 ms at 100 MHz
45 ) u_debounce (
46     .clk      (clk),
47     .rst      (rst),
48     .btn_in   (btn_send),
49     .pulse_out(start_pulse)
50 );
51
52 //-----
53 // 2. SPI master
54 //-----
55 wire [7:0] rx_data;
56 wire      busy, done;
57
58 spi_master #(
59     .CLK_DIV(50) // SPI clock = 1 MHz (100 MHz
60 / 100)
61 ) u_spi (
62     .clk      (clk),
63     .rst      (rst),
64     .start    (start_pulse),
65     .tx_data  (sw),
66     .rx_data  (rx_data),
67     .busy     (busy),
68     .done     (done),
69     .sclk     (ja_sclk),
70     .mosi     (ja_mosi),
71     .miso     (ja_miso),
72     .cs_n     (ja_cs_n)
73 );
74
75 //-----
76 // 3. Latch received byte from Arduino
77 //-----
78 reg [7:0] rx_latch;
79
80 always @(posedge clk) begin
81     if (rst)
82         rx_latch <= 8'h00;
83     else if (done)
84         rx_latch <= rx_data;
85 end
86
87 //-----
88 // 4. LED output
89 //-----
90 assign led[7:0]   = rx_latch;
91 assign led[14:8]  = 7'b000_0000;
92 assign led[15]    = busy;
93 endmodule

```

Listing 10.3: Top-Level Wrapper (top.v)

10.2.4 nexys4ddr.xdc

```

1  ##=====
2  ## nexys4ddr.xdc
3  ## Vivado pin/timing constraints -- Nexys 4 DDR
4  ## Device: XC7A100T-1CSG324C
5  ## Project: FPGA <-> Arduino SPI demo
6  ##=====
7
8  ##-----
9  ## 100 MHz system clock
10 ##-----
11 set_property PACKAGE_PIN E3 [get_ports clk]
12 set_property IOSTANDARD LVCMOS33 [get_ports clk]
13 create_clock -period 10.000 -name sys_clk_pin -waveform {0.000 5.000}
14 [get_ports clk]
15
16 ##-----
17 ## Buttons
18 ## BTNC (centre) -> rst (synchronous reset)
19 ## BTND (down) -> btn_send (trigger SPI transfer)
20 ##-----
21 set_property PACKAGE_PIN N17 [get_ports rst]
22 set_property IOSTANDARD LVCMOS33 [get_ports rst]
23
24 set_property PACKAGE_PIN P18 [get_ports btn_send]
25 set_property IOSTANDARD LVCMOS33 [get_ports btn_send]
26
27 ##-----
28 ## Switches SW[7:0] -- byte to send to Arduino
29 ##-----
30 set_property PACKAGE_PIN J15 [get_ports {sw[0]}]
31 set_property PACKAGE_PIN L16 [get_ports {sw[1]}]
32 set_property PACKAGE_PIN M13 [get_ports {sw[2]}]
33 set_property PACKAGE_PIN R15 [get_ports {sw[3]}]
34 set_property PACKAGE_PIN R17 [get_ports {sw[4]}]
35 set_property PACKAGE_PIN T18 [get_ports {sw[5]}]
36 set_property PACKAGE_PIN U18 [get_ports {sw[6]}]
37 set_property PACKAGE_PIN R13 [get_ports {sw[7]}]
38 set_property IOSTANDARD LVCMOS33 [get_ports {sw[*]}]
39
40 ##-----
41 ## LEDs [15:0]
42 ##-----
43 set_property PACKAGE_PIN H17 [get_ports {led[0]}]
44 set_property PACKAGE_PIN K15 [get_ports {led[1]}]
45 set_property PACKAGE_PIN J13 [get_ports {led[2]}]
46 set_property PACKAGE_PIN N14 [get_ports {led[3]}]
47 set_property PACKAGE_PIN R18 [get_ports {led[4]}]
48 set_property PACKAGE_PIN V17 [get_ports {led[5]}]
49 set_property PACKAGE_PIN U17 [get_ports {led[6]}]
50 set_property PACKAGE_PIN U16 [get_ports {led[7]}]
51 set_property PACKAGE_PIN V16 [get_ports {led[8]}]
52 set_property PACKAGE_PIN T15 [get_ports {led[9]}]
53 set_property PACKAGE_PIN U14 [get_ports {led[10]}]
54 set_property PACKAGE_PIN T16 [get_ports {led[11]}]
55 set_property PACKAGE_PIN V15 [get_ports {led[12]}]
56 set_property PACKAGE_PIN V14 [get_ports {led[13]}]
57 set_property PACKAGE_PIN V12 [get_ports {led[14]}]

```

```

57 set_property PACKAGE_PIN V11 [get_ports {led[15]}]
58 set_property IOSTANDARD LVCMOS33 [get_ports {led[*]}]
59
60 ##-----
61 ## Pmod JA -- SPI bus to Arduino
62 ##-----
63 set_property PACKAGE_PIN C17 [get_ports ja_cs_n]
64 set_property PACKAGE_PIN D18 [get_ports ja_mosi]
65 set_property PACKAGE_PIN E18 [get_ports ja_miso]
66 set_property PACKAGE_PIN G17 [get_ports ja_sclk]
67 set_property IOSTANDARD LVCMOS33 [get_ports ja_cs_n]
68 set_property IOSTANDARD LVCMOS33 [get_ports ja_mosi]
69 set_property IOSTANDARD LVCMOS33 [get_ports ja_miso]
70 set_property IOSTANDARD LVCMOS33 [get_ports ja_sclk]

```

Listing 10.4: Vivado Constraints File (nexys4ddr.xdc)

10.2.5 arduino_spi_slave.ino

```

1  /*=====
2   arduino_spi_slave.ino
3   Arduino Uno -- SPI hardware slave for FPGA demo
4
5   Protocol (each button press on FPGA):
6   1. FPGA (master) pulls CS_N LOW and sends 8 SPI clock pulses.
7   2. Both sides transfer simultaneously (full-duplex):
8       MOSI: FPGA sends SW[7:0]
9       MISO: Arduino sends the pre-loaded response
10  3. FPGA pulls CS_N HIGH -> SPI_STC interrupt fires on Arduino.
11  4. ISR reads the received byte, prints it over Serial,
12     and pre-loads SPDR with (received + 1) for the NEXT transfer.
13  5. FPGA latches what it received onto LED[7:0].
14
15  First-transfer note:
16  On power-up SPDR is pre-loaded with 0x01, so the VERY FIRST
17  transfer will show 0x01 on the FPGA LEDs regardless of SW.
18  Every subsequent transfer correctly shows (previous SW + 1).
19
20  Overflow:
21  0xFF + 1 wraps to 0x00 (uint8 arithmetic -- expected).
22
23  SPI Mode 0 (CPOL=0, CPHA=0), MSB first, 1 MHz -- matches FPGA.
24  =====*/
25
26  // ---- Shared state between ISR and loop() -----
27  volatile uint8_t g_received = 0;
28  volatile bool    g_newData  = false;
29
30  // ---- SPI Transfer Complete interrupt -----
31  ISR(SPI_STC_vect) {
32      g_received = SPDR; // read byte that FPGA sent
33      SPDR       = g_received + 1; // pre-load (received+1) for NEXT
34                          transfer
35      g_newData  = true;
36  }
37
38  void setup() {
39      Serial.begin(115200);

```

```

39
40 // Configure SPI pins for slave operation
41 pinMode(MISO, OUTPUT); // slave drives MISO
42 pinMode(MOSI, INPUT);  // master drives MOSI
43 pinMode(SCK, INPUT);   // master drives SCK
44 pinMode(SS, INPUT);    // master drives SS -- must be INPUT for
    slave
45
46 /*
47    SPCR register bits:
48    Bit 7 SPIE = 1 enable SPI interrupt
49    Bit 6 SPE  = 1 enable SPI peripheral
50    Bit 5 DORD = 0 MSB first (matches FPGA)
51    Bit 4 MSTR = 0 slave mode
52    Bit 3 CPOL = 0 clock idles LOW (Mode 0)
53    Bit 2 CPHA = 0 sample on rising edge (Mode 0)
54 */
55 SPCR = (1 << SPE) | (1 << SPIE); // 0b11000000
56
57 SPDR = 0x01; // first response pre-loaded
58
59 sei(); // enable global interrupts
60
61 Serial.println(F
62 ("====="));
63 Serial.println(F(" Arduino SPI Slave -- FPGA demo"));
64 Serial.println(F(" Set SW[7:0] on FPGA and press BTND."));
65 Serial.println(F(" LEDs show what Arduino sent back (SW+1)."));
66 Serial.println(F(" NOTE: first LED result = 0x01 (normal)."));
67 Serial.println(F
68 ("====="));
69 Serial.println(F(" RECEIVED (FPGA) SENT BACK (FPGA LEDs)"));
70 Serial.println(F
71 ("-----"));
72 }
73
74 void loop() {
75     if (g_newData) {
76         noInterrupts();
77         uint8_t rx = g_received;
78         g_newData = false;
79         interrupts();
80
81         uint8_t tx = (uint8_t)(rx + 1);
82
83         Serial.print(F(" 0x"));
84         if (rx < 0x10) Serial.print('0');
85         Serial.print(rx, HEX);
86         Serial.print(F(" (dec "));
87         if (rx < 100) Serial.print(' ');
88         if (rx < 10) Serial.print(' ');
89         Serial.print(rx, DEC);
90         Serial.print(F(")"));
91
92         Serial.print(F("          0x"));
93         if (tx < 0x10) Serial.print('0');
94         Serial.print(tx, HEX);
95         Serial.print(F(" (dec "));
96         if (tx < 100) Serial.print(' ');
97         if (tx < 10) Serial.print(' ');
98         Serial.print(tx, DEC);
99         Serial.print(F(")"));
100     }

```

```

93         if (tx < 100) Serial.print(' ');
94         if (tx < 10)  Serial.print(' ');
95         Serial.print(tx, DEC);
96         Serial.println(F(""));
97     }
98 }

```

Listing 10.5: Arduino SPI Slave Sketch (arduino_spi_slave.ino)

10.2.6 tb_spi_system.v

```

1  //=====
2  // tb_spi_system.v
3  // Testbench for spi_master.v
4  // Verified compatible with Vivado 2022.1 (Behavioral Simulation)
5  //
6  // Tests:
7  //   Test 1 - Reset clears all outputs
8  //   Test 2 - Normal transaction 0x05 -> slave returns 0x06
9  //   Test 3 - Normal transaction 0xA3 -> slave returns 0x5C
10 //   Test 4 - Boundary: tx=0x00, miso=0xFF (all bits inverted)
11 //   Test 5 - Boundary: tx=0xFF, miso=0x00 (all bits inverted)
12 //   Test 6 - Back-to-back transaction 0x12 -> 0x34
13 //   Test 7 - done pulse is exactly 1 clock cycle wide
14 //=====
15 `timescale 1ns / 1ps
16
17 module tb_spi_system;
18
19     //-----
20     // DUT signals
21     //-----
22     reg        clk;
23     reg        rst;
24     reg        start;
25     reg [7:0]  tx_data;
26     wire [7:0] rx_data;
27     wire       busy;
28     wire       done;
29     wire       sclk;
30     wire       mosi;
31     reg        miso;
32     wire       cs_n;
33
34     // Testbench state
35     reg [7:0]  miso_byte;
36     reg [7:0]  captured_mosi;
37     integer    pass_count;
38     integer    fail_count;
39     integer    i;
40     reg        done_seen;
41
42     //-----
43     // DUT - CLK_DIV=4 for fast simulation
44     //-----
45     spi_master #(
46         .CLK_DIV(4)
47     ) dut (

```

```

48     .clk      (clk),
49     .rst      (rst),
50     .start    (start),
51     .tx_data  (tx_data),
52     .rx_data  (rx_data),
53     .busy     (busy),
54     .done     (done),
55     .sclk     (sclk),
56     .mosi     (mosi),
57     .miso     (miso),
58     .cs_n     (cs_n)
59 );
60
61 //-----
62 // Clock - 10 ns period (100 MHz)
63 //-----
64 initial clk = 0;
65 always #5 clk = ~clk;
66
67 //-----
68 // Task: send one byte, drive MISO, check MOSI + rx_data
69 //-----
70 task run_transaction;
71     input [7:0] tx;
72     input [7:0] rx;
73     begin
74         miso_byte      = rx;
75         tx_data        = tx;
76         captured_mosi  = 8'h00;
77
78         @(posedge clk); #1;
79         start = 1;
80         @(posedge clk); #1;
81         start = 0;
82
83         @(posedge clk); #1;
84         if (cs_n != 1'b0) begin
85             $display("FAIL CS_N did not assert tx=0x%02X", tx);
86             fail_count = fail_count + 1;
87         end
88
89         for (i = 7; i >= 0; i = i - 1) begin
90             @(posedge sclk); #1;
91             miso = miso_byte[i];
92             @(negedge sclk); #2;
93             captured_mosi[i] = miso;
94         end
95
96         @(posedge done);
97
98         if (captured_mosi != tx) begin
99             $display("FAIL MOSI tx=0x%02X captured=0x%02X",
100 tx, captured_mosi);
101             fail_count = fail_count + 1;
102         end else begin
103             $display("PASS MOSI tx=0x%02X transmitted
correctly", tx);
            pass_count = pass_count + 1;

```

```

104         end
105
106         @(posedge clk); #1;
107         if (rx_data != rx) begin
108             $display("FAIL MIS0 expected rx=0x%02X got
rx_data=0x%02X", rx, rx_data);
109             fail_count = fail_count + 1;
110         end else begin
111             $display("PASS MIS0 rx_data=0x%02X received
correctly", rx_data);
112             pass_count = pass_count + 1;
113         end
114
115         if (cs_n != 1'b1) begin
116             $display("FAIL CS_N did not deassert after transfer
tx=0x%02X", tx);
117             fail_count = fail_count + 1;
118         end
119
120         if (busy != 1'b0) begin
121             $display("FAIL busy still HIGH after done tx=0x%02X
", tx);
122             fail_count = fail_count + 1;
123         end
124
125         repeat(10) @(posedge clk);
126     end
127 endtask
128
129 //-----
130 // Task: check done pulse is exactly 1 clock wide
131 //-----
132 task check_done_width;
133     input [7:0] tx;
134     input [7:0] rx;
135     begin
136         miso_byte = rx;
137         tx_data = tx;
138
139         @(posedge clk); #1;
140         start = 1;
141         @(posedge clk); #1;
142         start = 0;
143
144         for (i = 7; i >= 0; i = i - 1) begin
145             @(posedge sclk); #1;
146             miso = miso_byte[i];
147             @(negedge sclk);
148         end
149
150         @(posedge done); #1;
151
152         @(posedge clk); #1;
153         if (done != 1'b0) begin
154             $display("FAIL done pulse wider than 1 clock cycle")
;
155             fail_count = fail_count + 1;
156         end else begin

```

```

157         $display("PASS   done pulse width = exactly 1 clock
cycle");
158         pass_count = pass_count + 1;
159     end
160
161     repeat(10) @(posedge clk);
162 end
163 endtask
164
165 //-----
166 // Stimulus
167 //-----
168 initial begin
169     rst           = 1;
170     start         = 0;
171     tx_data       = 8'h00;
172     miso          = 0;
173     miso_byte     = 8'h00;
174     captured_mosi = 8'h00;
175     pass_count    = 0;
176     fail_count    = 0;
177     done_seen     = 0;
178     i             = 0;
179
180     $display("=====");
181 ;
182     $display("  tb_spi_system  Vivado 2022.1 Behavioral Sim  ");
183     $display("  CLK_DIV=4   10 ns clock  SPI clock = 40 ns  ");
184     $display("=====");
185 ;
186
187     repeat(5) @(posedge clk);
188
189     // TEST 1: reset state
190     if (cs_n !== 1'b1 || sclk !== 1'b0 ||
191         mosi !== 1'b0 || busy !== 1'b0 || done !== 1'b0) begin
192         $display("FAIL   Test 1: outputs not cleared in reset");
193         fail_count = fail_count + 1;
194     end else begin
195         $display("PASS   Test 1: all outputs correct during reset
196 ");
197         pass_count = pass_count + 1;
198     end
199
200     @(posedge clk); #1;
201     rst = 0;
202     repeat(5) @(posedge clk);
203
204     $display("-----");
205 ;
206     $display("  Test 2: tx=0x05  miso=0x06");
207     run_transaction(8'h05, 8'h06);
208
209     $display("-----");
210 ;
211     $display("  Test 3: tx=0xA3  miso=0x5C");
212     run_transaction(8'hA3, 8'h5C);
213
214

```



```

209     $display("-----")
210 ;
211     $display("  Test 4: boundary tx=0x00  miso=0xFF");
212     run_transaction(8'h00, 8'hFF);
213
214     $display("-----")
215 ;
216     $display("  Test 5: boundary tx=0xFF  miso=0x00");
217     run_transaction(8'hFF, 8'h00);
218
219     $display("-----")
220 ;
221     $display("  Test 6: back-to-back tx=0x12  miso=0x34");
222     run_transaction(8'h12, 8'h34);
223
224     $display("-----")
225 ;
226     $display("  Test 7: done pulse width  tx=0xAA  miso=0x55");
227     check_done_width(8'hAA, 8'h55);
228
229     $display("=====")
230 ;
231     $display("  RESULTS:  %0d PASSED      %0d FAILED",
232               pass_count, fail_count);
233     if (fail_count == 0)
234         $display("    *** ALL TESTS PASSED ***");
235     else
236         $display("    *** %0d FAILURE(S) - CHECK WAVEFORM ***",
237               fail_count);
238     $display("=====")
239 ;
240     $finish;
241 end
242
243 initial begin
244     #200000;
245     $display("TIMEOUT - 200 us exceeded, FSM may be stuck");
246     $finish;
247 end
248 endmodule

```

Listing 10.6: Self-Checking Testbench (tb_spi_system.v)